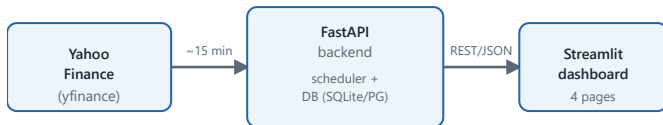




1 • Project Overview

FinPulse tracks 20 NSE-listed Indian companies across 10 sectors. A FastAPI backend pulls live price and fundamental data from Yahoo Finance on a schedule and stores it in a database; a Streamlit frontend reads that data through the backend's REST API and presents it as an interactive dashboard — historical charts, fundamentals, cross-company comparison, and a sector view.

2 • Project Architecture



The backend refreshes data on a background schedule and is the only component that talks to Yahoo Finance or the database. The frontend only calls the backend's REST API, so the two halves deploy and scale independently.

3 • APIs Used

- **Yahoo Finance** (via `yfinance`) — live quotes, fundamentals (P/E, EPS), historical OHLCV. No key required.
- **FinPulse's own REST API** — 5 endpoints, consumed only by the Streamlit frontend (see §4 for the list).

4 • Database Design

- `companies` — ticker, name, sector (static reference)
- `stock_snapshot` — latest price/cap/P-E/EPS per ticker, upserted every refresh
- `price_history` — daily OHLCV, unique per (ticker, date), backfilled once (1y)
- SQLite in the current deployment; swaps to Postgres via `DATABASE_URL`

Endpoints: `/stocks`, `/stocks/{ticker}`,
`/stocks/{ticker}/history`, `/market-summary`, `/refresh`

7 • Future Improvements

- WebSocket/SSE push instead of cache-based polling for real-time updates
- Additional ratios: ROE, debt/equity, dividend yield
- User-defined watchlists instead of a fixed 20-company universe
- Email/Telegram alerts on price or valuation thresholds
- Move the deployed DB from SQLite to Postgres (pooler URL)

5 • Features Implemented

- Live + historical data for 20 companies (price, market cap, P/E, EPS, volume, day change)
- 5 REST endpoints (3 were required)
- 4-page dashboard: Overview, Company Detail, Comparison, Sector View
- Candlestick chart with a direction-colored volume overlay
- Custom stock screener (price / P-E / market-cap range sliders)
- Dark mode by default

6 • Challenges Faced

- `yfinance`'s `fast_info.get()` only recognizes camelCase keys — it silently returned `None` for every field until caught by inspecting live responses; fixed with attribute access instead.
- Supabase's direct Postgres connection is IPv6-only and unreachable from the deploy network. Shipped on SQLite instead — reasonable here since the app re-seeds all data from `yfinance` on every startup.
- Avoided fetching all 20 tickers per page load (slow, risks Yahoo's rate limits) — data is cached and refreshed on a fixed schedule instead.

GitHub: github.com/shouryacha/FinPulse Frontend: finpulse-hdstc3fnmwdvitbyppb5jq.streamlit.app

Backend API: finpulse-api-5t6h.onrender.com